

An Exact Penalty Matrix for Cubic Smoothing Splines on Arbitrary Knot Vectors

Aadya Chinubhai

August 3, 2026

Abstract

These notes derive the penalty matrix Ω of a cubic smoothing spline from first principles, for an arbitrary knot vector. Starting from the definition of the B-spline basis, we express differentiation as a matrix, integrate the squared second derivative in closed form, and arrive at an exact factorization $\Omega = C^\top R C$ built from the knot vector alone.

Contents

1 The Basics	2
2 Differentiation as a matrix	3
3 Integrating the squared second derivative: hat functions	4
4 The penalty matrix	6
5 Using Ω: the penalized solve	7
6 Boundary conditions	7
7 Higher-degree splines: what would change	8
8 What is from the literature, and what is derived here	9
9 Validation	9
10 Conditioning at large λ	10
11 Related software	11

1 The Basics

A cubic spline on a knot vector t is a linear combination of cubic B-spline basis functions,

$$f(u) = \sum_{j=1}^m c_j B_j(u), \quad (1)$$

where B_j are the basis functions, $\mathbf{c} = (c_1, \dots, c_m)^\top$ are the coefficients, and m is the number of basis functions supported by t .

The roughness penalty is the integrated squared curvature of f ,

$$\int (f''(u))^2 du, \quad (2)$$

which, substituting (1), is a quadratic form in the coefficients: $\mathbf{c}^\top \Omega \mathbf{c}$ with

$$\Omega_{ij} = \int B_i''(u) B_j''(u) du. \quad (3)$$

Computing Ω exactly, for an arbitrary knot vector t , is the subject of this document.

The boundary knots are repeated four times (the order of the spline). The repeats leave zero width between them, so the first basis function has no room to ramp up the way an interior one does, and, since each repetition also removes one continuity requirement at that point, nothing forces it to. Its ramp compresses to nothing and the basis function simply begins at a single point, at full height: $B_1(t_{\min}) = 1$, with every other basis function zero there. The spline is therefore pinned to its first coefficient, $f(t_{\min}) = c_1$, and likewise $f(t_{\max}) = c_m$ at the right end.

Symbols used in this document

Before deriving anything, here is every symbol that appears, in order of first use. The running example throughout is the knot vector $t = (0, 0, 0, 0, 0.7, 2.3, 3, 3, 3, 3)$.

symbol	what it is	size (example)
t	knot vector, boundary knots repeated $4\times$	10 knots
K	number of interior knots	2
$m = K + 4$	number of cubic basis functions	6
u	the free variable the spline is a function of	,
$B_j(u)$	j -th cubic B-spline basis function	m of them
$f(u)$	the spline, $f = \sum_j c_j B_j$	,
$\mathbf{c}$	coefficients of f in the cubic basis	$m = 6$
Ω	penalty matrix, $\Omega_{ij} = \int B_i'' B_j''$	$m \times m$
D_1	differentiation matrix, cubic $\rightarrow$ quadratic coeffs	7×6
D_2	differentiation matrix, quadratic $\rightarrow$ linear coeffs	8×7
$C = D_2 D_1$	second-derivative matrix, $\mathbf{c} \rightarrow$ coeffs of f''	8×6
$N_p(u)$	p -th hat function (linear B-spline)	$m + 2 = 8$ of them
$\boldsymbol{\gamma} = C \mathbf{c}$	coefficients of f'' in the hat basis	$m + 2 = 8$
R	hat mass matrix, $R_{pq} = \int N_p N_q$	8×8 , tridiagonal
h, h'	widths of knot intervals	,
<i>Used only in the solve and validation:</i>		
$(x_i, y_i), w_i$	the n data points and their weights	n each
X	design matrix, $X_{ij} = B_j(x_i)$	$n \times m$
W	diagonal matrix of the weights	$n \times n$
λ	smoothing parameter	scalar
$\mathbf{g}$	Greville points, $g_j = (t_{j+1} + t_{j+2} + t_{j+3})/3$	m

The logic of the document in one line: Section 2 builds C , Section 3 builds R , and Section 4 combines them into $\Omega = C^\top R C$.

2 Differentiation as a matrix

Start from what a cubic spline is on one interval: an ordinary cubic polynomial $au^3 + bu^2 + cu + d$. Differentiate it once and you get a quadratic; differentiate again and you get a straight line. This stays true piece by piece across the whole spline: *the derivative of a cubic spline is a quadratic spline, and its second derivative is a piecewise-linear function*, and both live on the same knots, since differentiation does not move the break points.

Now, our spline is not written as a, b, c, d per piece, it is written as coefficients $c_1, \dots, c_6$ in the B-spline basis (eq. (1)). So the question becomes: if I know the six numbers describing f , what are the seven numbers describing f' in the quadratic B-spline basis? (Seven, because our ten knots carry six cubic but seven quadratic basis functions, the counting table of Section 2.)

De Boor's formula (de Boor, 2001, p. 117) answers this: each new coefficient is a *slope* computed from two neighbouring old ones,

$$\text{new coefficient } j = 3 \cdot \frac{c_j - c_{j-1}}{t_{j+3} - t_j}. \quad (4)$$

Read it like: $c_j - c_{j-1}$ is a rise between neighbouring control heights, $t_{j+3} - t_j$ is the horizontal distance those two coefficients are anchored over, and the factor 3 is the same 3 as in $(u^3)' = 3u^2$.

Since every new coefficient is just a weighted difference of two old ones, the whole operation is a matrix. Row j of that matrix picks out c_{j-1} and c_j with weights $\mp d_j$, where $d_j = 3/(t_{j+3} - t_j)$.

For our ten knots $t = (0, 0, 0, 0, 0.7, 2.3, 3, 3, 3, 3)$ the matrix is 7×6 (in general $(m+1) \times m$), six coefficients in, seven out:

$$D_1 = \begin{pmatrix} 0 & 0 & 0 & 0 & 0 & 0 \\ -4.29 & 4.29 & 0 & 0 & 0 & 0 \\ 0 & -1.30 & 1.30 & 0 & 0 & 0 \\ 0 & 0 & -1 & 1 & 0 & 0 \\ 0 & 0 & 0 & -1.30 & 1.30 & 0 \\ 0 & 0 & 0 & 0 & -4.29 & 4.29 \\ 0 & 0 & 0 & 0 & 0 & 0 \end{pmatrix}$$

For instance the 4.29 in the second row is $3/(t_5 - t_2) = 3/0.7$. The first and last rows are zero because their run $t_{j+3} - t_j$ is the distance across the repeated boundary knots, which is zero. There is no width for a slope to live on; this is the clamping of Section 2 seen from the coefficient side.

Doing the same again (quadratics $\rightarrow$ lines, with a 2 in place of the 3 and a two-knot run in the denominator) gives an 8×7 (in general $(m+2) \times (m+1)$) matrix D_2 , and chaining the two

$$C = D_2 D_1 \in \mathbb{R}^{(m+2) \times m} \quad (8 \times 6 \text{ here}), \quad (5)$$

takes the six numbers describing f straight to the eight numbers describing f'' as a piecewise-linear function, in the basis of *hat functions* on the same knots:

$$C = \begin{pmatrix} 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \\ 12.24 & -15.97 & 3.73 & 0 & 0 & 0 \\ 0 & 1.13 & -2.00 & 0.87 & 0 & 0 \\ 0 & 0 & 0.87 & -2.00 & 1.13 & 0 \\ 0 & 0 & 0 & 3.73 & -15.97 & 12.24 \\ 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \end{pmatrix}$$

Each row of C combines *three* neighbouring coefficients with alternating signs, a second difference, which is exactly what you would expect from a matrix whose job is to take a second derivative.

Two things to notice, because the whole construction rests on them. First, C was built from the knot vector alone: no data point x_i or y_i appears anywhere in this section. Second, everything was exact, eq. (4) is an identity, not an approximation.

3 Integrating the squared second derivative: hat functions

What kind of function is f'' ?

Go back to a single piece of the spline, the cubic $au^3 + bu^2 + cu + d$. Differentiate twice: $6au + 2b$, a straight line. So on every knot interval, f'' is a straight line; and since f is a cubic spline, f'' is continuous across the knots. A function that is straight on each interval and continuous at the joints is a connect the dots polyline, completely determined by its values at the knots.

Polylines have their own natural basis: the *hat functions*. The hat N_p is the polyline equal to 1 at knot p and 0 at every other knot, a triangle, rising over one interval and falling over the next. Every polyline is a combination of hats, with its knot-values as the coefficients. And hats are not a new ingredient: they are exactly the *linear* B-splines on our knot vector (the three-knot windows of Section 2). That is why the previous section could deliver f'' directly in this basis: with $\gamma = Cc$,

Why can any polyline be written in hats, and what are the coefficients?

Hats have one special property: at any knot, exactly one of them is nonzero, and its value there is 1: $N_p(t_q) = 1$ if $p = q$ and 0 otherwise. Take the combination whose coefficients are the polyline's own values at the knots, $g = \sum_p f''(t_p) N_p$, and evaluate it at a knot t_q : every hat vanishes except N_q , which contributes $f''(t_q) \cdot 1$. So g agrees with f'' at every knot; both are polylines, and a polyline is determined by its knot values, hence they are the same function:

$$f''(u) = \sum_{p=1}^{m+2} \gamma_p N_p(u), \quad \gamma_p = f''(t_p), \quad \gamma = C\mathbf{c} \in \mathbb{R}^{m+2}. \quad (6)$$

The hat basis is *interpolatory*: its coefficients are function values. (Contrast the cubic basis, where the c_j are control heights, not values of f .) This also says what the entries of $\gamma = C\mathbf{c}$ mean: the second derivative of the spline, evaluated at the knots.

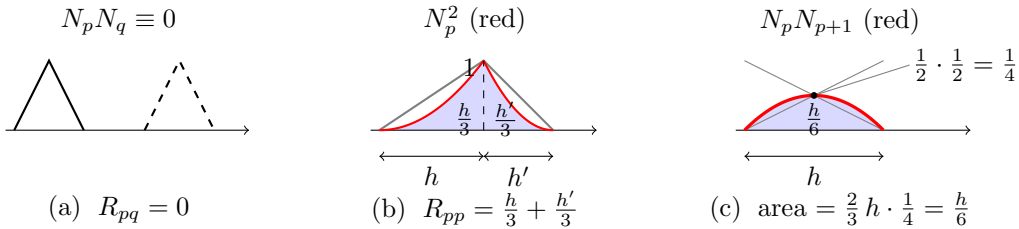
Squaring and integrating

Expand the square of the sum in the penalty (2):

$$\int (f'')^2 du = \sum_p \sum_q \gamma_p \gamma_q \underbrace{\int N_p(u) N_q(u) du}_{R_{pq}} = \gamma^\top R \gamma, \quad R \in \mathbb{R}^{(m+2) \times (m+2)}. \quad (7)$$

One big integral has become a table of small ones, $R_{pq} = \int N_p N_q$, called the *mass matrix* of the hat basis. The table is almost empty: two hats that are not neighbours share no ground, their product is zero everywhere, and $R_{pq} = 0$. Only two kinds of entry survive, a hat with itself, and a hat with its next-door neighbour, so R is tridiagonal.

The three cases, and two parabola areas



Case (a) is the sparsity of R : hats that are not neighbours share no ground, their product vanishes identically, and $R_{pq} = 0$. Only two kinds of entry survive, and both are areas under parabolas, straight sides multiplied by straight sides give quadratics.

Case (b), *a hat with itself*. On its rising interval of width h the hat is the ramp u/h , so N_p^2 is a parabola climbing from 0 to 1 with its vertex at the start. The area under such a parabola is one third of its bounding rectangle,

$$\frac{1}{3} \times h \times 1 = \frac{h}{3},$$

and the falling interval (width h') contributes $h'/3$ the same way:

$$R_{pp} = \frac{h + h'}{3} = \frac{t_{p+2} - t_p}{3}. \quad (8)$$

Case (c), a hat with its neighbour. They overlap on one interval, one falling as the other rises. Their product is zero at both ends and peaks at the midpoint, where both hats equal $\frac{1}{2}$: a parabolic arch of base h and height $\frac{1}{4}$. By Archimedes' rule the area under a parabolic arch is two thirds of its bounding rectangle,

$$\frac{2}{3} \times h \times \frac{1}{4} = \frac{h}{6},$$

so

$$R_{p,p+1} = \frac{t_{p+2} - t_{p+1}}{6}. \quad (9)$$

That is the entire computation of R : one third-rule and one two-thirds rule. Nothing is approximate, and no data has entered, both entries are read off the knot spacings.

The running example

Our ten knots carry 8 hats, so R is 8×8 . But the first two and last two hats are built on windows of repeated boundary knots (widths $t_{p+2} - t_p = 0$): they are identically zero, and (8)–(9) produce zero rows for them automatically, with no special-casing. These zero rows match the zero rows of C in Section 2, they are coefficients of hats that do not exist. With interval widths 0.7, 1.6, 0.7:

$$R = \begin{pmatrix} 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0.2333 & 0.1167 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0.1167 & 0.7667 & 0.2667 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0.2667 & 0.7667 & 0.1167 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0.1167 & 0.2333 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \end{pmatrix}$$

Spot-check one entry: $R_{44} = (t_6 - t_4)/3 = 2.3/3 = 0.7667$, the widest hat's width over three. And a global check: since the hats sum to one everywhere, row p of R must sum to $\int N_p = (t_{p+2} - t_p)/2$, half the hat's width. The row sums here are 0.35, 1.15, 1.15, 0.35 for the four live hats, exactly the half-widths.

The section's result, in one line: by (7), integrating the squared second derivative is a matrix sandwich, $\int (f'')^2 = \gamma^\top R \gamma$, with R known exactly from the knots alone. The next section only has to substitute $\gamma = C\mathbf{c}$.

4 The penalty matrix

Sections 2 and 3 each produced one half of the computation: the matrix C turns the spline's coefficients into the coefficients of its second derivative, $\gamma = C\mathbf{c}$, and the matrix R integrates the square of anything written in hat coefficients, $\int (f'')^2 = \gamma^\top R \gamma$. Substituting the first into the second eliminates γ :

$$\int (f'')^2 du = (C\mathbf{c})^\top R (C\mathbf{c}) = \mathbf{c}^\top (C^\top R C) \mathbf{c}, \quad (10)$$

so, comparing with (3),

$$\boxed{\Omega = C^\top R C} \quad (11)$$

5 Using Ω : the penalized solve

The smoothing problem the implementation solves is

$$\min_{\mathbf{c} \in \mathbb{R}^m} (\mathbf{y} - X\mathbf{c})^\top W(\mathbf{y} - X\mathbf{c}) + \lambda \mathbf{c}^\top \Omega \mathbf{c}, \quad (12)$$

where $X \in \mathbb{R}^{n \times m}$, $X_{ij} = B_j(x_i)$, is the design matrix (n data points, m coefficients, rectangular, since $n \neq m$ in general), and $W = \text{diag}(w) \in \mathbb{R}^{n \times n}$ holds the weights. Setting the gradient to zero gives the normal equations

$$(X^\top W X + \lambda \Omega) \mathbf{c} = X^\top W \mathbf{y}, \quad (13)$$

an $m \times m$ system. The matrix on the left has three properties that together select the solver:

- **Symmetric** (real Hermitian): $X^\top W X$ is symmetric by construction, and $\Omega = C^\top R C$ is symmetric because R is.
- **Positive definite** for $\lambda > 0$. Both terms are positive semi-definite because both quadratic forms are sums or integrals of squares: $\mathbf{c}^\top X^\top W X \mathbf{c} = \sum_i w_i (f(x_i))^2 \geq 0$, and $\mathbf{c}^\top \Omega \mathbf{c} = \int (f'')^2 \geq 0$. Now suppose $\mathbf{c}$ is a null vector of the sum. Since neither term can be negative, both must vanish. The second vanishing, $\int (f'')^2 = 0$, forces $f'' \equiv 0$: the spline is a straight line. (This is also why $\Omega \mathbf{1} = 0$ and $\Omega \mathbf{g} = 0$ for the Greville vector $\mathbf{g}_j = (t_{j+1} + t_{j+2} + t_{j+3})/3$, the coefficient vectors of the constant and linear functions.) The first vanishing then forces this line to be zero at every data point x_i ; a straight line with two or more distinct roots is identically zero, so $\mathbf{c} = 0$. Hence, for any data set with at least two distinct x_i , the system matrix has no nonzero null vector: it is positive definite.
- **Banded** with bandwidth 3: cubic B-splines i and j have disjoint supports for $|i - j| > 3$, so both $X^\top W X$ and Ω are 7-banded.

A symmetric positive-definite banded system is solved by banded Cholesky factorization: in `scipy`, `solveh_banded` (the `h` is for Hermitian), which accepts only the $4 \times m$ unique lower bands and does roughly half the work of general banded LU. (The existing `t = x` implementation uses the general solver `solve_banded` because it poses the problem in pre-divided form, $(X + \lambda wE)\mathbf{c} = \mathbf{y}$, the normal equations with $X^\top W$ cancelled from the left, where wE denotes its penalty term in the natural-spline basis, and that matrix is not symmetric. The same problem posed as normal equations *is* symmetric and could equally use the Cholesky solver, as the new path in fact does when $t = x$.)

6 Boundary conditions

Nothing in the construction so far imposes any behaviour at the ends of the interval: the minimization runs over all cubic splines on t , and the boundary does whatever the objective prefers. This section records how *natural* boundary conditions, zero curvature at both ends,

$$f''(t_{\min}) = 0, \quad f''(t_{\max}) = 0 \quad (14)$$

, fit into the same framework, should one wish to impose them.

The key observation is that (14) is already expressed in our matrices. By the interpolatory property (6), the entries of $\boldsymbol{\gamma} = C\mathbf{c}$ are the values of f'' at the knots, so (14) says simply that the first and last live entries of $C\mathbf{c}$ vanish: two linear constraints on $\mathbf{c}$, each involving only the two or three boundary rows of C that are nonzero there.

Linear constraints are imposed by *null-space reduction*. Let $Z \in \mathbb{R}^{m \times (m-2)}$ be a matrix whose columns span the coefficient vectors satisfying both constraints. Substituting $\mathbf{c} = Z\tilde{\mathbf{c}}$ turns the normal equations (13) into an $(m-2) \times (m-2)$ system in the reduced coefficients:

$$Z^\top (X^\top W X + \lambda \Omega) Z \tilde{\mathbf{c}} = Z^\top X^\top W \mathbf{y}, \quad (15)$$

after which $\mathbf{c} = Z\tilde{\mathbf{c}}$ recovers the B-spline coefficients. Because each constraint touches only a few boundary coefficients, Z differs from the identity only in its first and last few rows, and the reduced system remains banded.

Two remarks. First, this is not hypothetical infrastructure: the existing $t = x$ implementation in `scipy` is this construction. It works directly in a natural-spline basis of dimension $n = m - 2$, and its basis-change map (eqs. (2.2.10) of Zemlyanoy (2022)) is exactly such a Z ; we verified numerically, to 13 significant digits, that its penalty satisfies $X^\top W (wE) = Z^\top \Omega Z$ with Ω from (11), the existing penalty is ours, restricted to the natural subspace.

Second, the distinction between imposing and obtaining. At $t = x$, imposing (14) changes nothing: as shown in Section 9, the unconstrained minimizer satisfies the natural conditions on its own. For general knots $t \neq x$ this is no longer true, the constraint genuinely alters the solution, chiefly its behaviour near and beyond the boundaries (a natural spline extrapolates linearly; an unconstrained one leaves the interval as a full cubic).

7 Higher-degree splines: what would change

Suppose we wanted splines of degree $k > 3$ with the same penalty $\int (f'')^2$. Walking through the construction section by section, almost nothing changes, and the one thing that does has a clean, still-exact replacement.

What survives, and why. De Boor’s formula (4) holds at every order, so the differentiation matrices, and hence C , now two reductions starting from degree k , are built exactly as in Section 2. The assembly $\Omega = C^\top R C$ never used “cubic” anywhere. The null space of Ω is still exactly the straight lines (lines have zero curvature at every degree), so the positive-definiteness argument of Section 5 and the invariant tests $\Omega \mathbf{1} = 0$, $\Omega \mathbf{g} = 0$ carry over verbatim. Bandedness survives too, with wider bands: basis functions overlap $|i - j| \leq k$ neighbours, so Ω and $X^\top W X$ are $(2k + 1)$ -banded and `solveh_banded` applies unchanged. Even the oracle generalizes: `fda::bsplinepen` takes the order as an argument.

What breaks: the parabola shortcut. After two derivatives, f'' now lands in the degree- $(k - 2)$ basis, not the hats. The products $N_p N_q$ are no longer parabolas, so the third-rule and two-thirds-rule of Section 3, the entire “computation” of R , no longer apply, and no comparably short formula exists.

The replacement: quadrature that is secretly exact. On each knot interval, $N_p N_q$ is a single polynomial of known degree $2(k - 2)$. A Gauss–Legendre rule with g points integrates any polynomial of degree $\leq 2g - 1$ with zero error, so choosing $g = k - 1$ points per interval,

$$R = E^\top \text{diag}(w) E \quad E_{\ell p} = N_p(u_\ell), \quad (16)$$

with u_ℓ, w_ℓ the Gauss points and weights of all intervals, reproduces every entry $\int N_p N_q$ exactly, the same degree-matching logic as Simpson’s rule in Wand and Ormerod (2008), one rung up the ladder. The cost is one basis evaluation per point, $O(n)$ total, and the construction remains data-free. Indeed the cubic case is this same idea done once by hand: Archimedes’ rule is the exact quadrature of a parabola, and (8)–(9) are its pre-computed answers.

In short: for $k > 3$, replace two hand-derived numbers by a $(k - 1)$ -point exact quadrature inside R , and touch nothing else. This is recorded here as a design note; the implementation is deliberately cubic-only.

8 What is from the literature, and what is derived here

The problem itself is classical. The penalized objective and the penalty matrix $\Omega_{ij} = \int B_i'' B_j''$ are standard in smoothing-spline theory (Wahba, 1990); using a B-spline basis with freely chosen knots and this exact integral penalty is known in the statistics literature as *O’Sullivan penalized splines* (Wand and Ormerod, 2008).

Ramsay and Silverman (2005, Ch. 5) discuss computing Ω and state that for B-spline bases the integrals “can be computed analytically,” but decline to show how: “the details (see de Boor, 2002) are intricate, and few users of FDA will want to write programming code for this problem,” referring readers instead to their software packages. The remainder of their section concerns numerical quadrature fallbacks. In other words, the standard reference names the gap this document fills, without filling it.

One prior exact construction is published. Wand and Ormerod (2008, eq. (3)) obtain Ω by observing that each $B_i'' B_j''$ is piecewise quadratic, so Simpson’s rule with three evaluations per knot interval is *exact*: their $\Omega = (\tilde{B}'')^\top \text{diag}(w) \tilde{B}''$ built from values of B'' at interval endpoints and midpoints. The present construction reaches the same matrix by a different route, coefficient algebra rather than evaluation: no quadrature nodes appear, and the banded structure and null space of Ω are visible directly in the factors C and R .

Equation by equation, the provenance is:

- **From the literature:** the derivative formula (4) (de Boor, 2001, p. 117); the hat-function integrals (8) and (9) (classical finite-element mass-matrix entries, rederived in Section 3 for completeness); the definition of the penalty (3) (Wahba, 1990).
- **Derived in this document:** the matrix form of differentiation (D_1 , D_2) including the treatment of repeated knots; the composition (5) expressing f'' in the hat basis; and the assembly $\Omega = C^\top R C$, eq. (11). These steps were derived independently from (4); the `fda` package of Ramsay & Silverman implements an analytic Ω , but its source is GPL-licensed and was deliberately not consulted, only its numerical output was used, as a black-box test.

9 Validation

All checks are collected in a companion repository,

<https://github.com/aadya940/scipy-bspline-testing>

whose scripts are annotated with the equation numbers of this document and fail on any mismatch. The checks, and what each one certifies:

Check	Compared against	Result	Certifies
Ω on irregular knots	<code>fda:bsplinepen</code> (R)	10^{-6}	eq. (11)
Ω on irregular knots	direct quadrature of (3)	10^{-12}	eq. (11)
R closed form	quadrature of $\int N_p N_q$	10^{-10}	eqs. (8), (9)
Ω via Simpson exactness	Wand and Ormerod (2008, eq. (3))	10^{-8}	independent route
$\Omega \mathbf{1} = 0$	-	10^{-12}	null space = lines
symmetry, bandwidth 3	-	exact	structure of Ω
full solve, $t = \text{clamped } x$	existing <code>make_smoothing_spline</code>	10^{-14} – 10^{-9}	entire pipeline

Two end-to-end checks exercise the full solve path in the proposed `scipy` implementation. At $\lambda = 0$ with sparse knots, the smoothing spline reproduces `make_lsqr_spline` to 10^{-15} .

The second check places the knots at the clamped data sites, $t = x$, and compares against the *existing* `make_smoothing_spline`. To state precisely what is being tested, write the objective as

$$J(f) = \sum_{i=1}^n w_i (y_i - f(x_i))^2 + \lambda \int (f'')^2 du, \quad (17)$$

and consider three nested sets of candidate functions:

$$\mathcal{S}_{\text{nat}} \subset \mathcal{S}_{\text{spl}} \subset \mathcal{F}, \quad (18)$$

where $\mathcal{F}$ is all twice-differentiable functions, $\mathcal{S}_{\text{spl}}$ is the cubic splines on the knots $t = x$ (dimension $n + 2$), and $\mathcal{S}_{\text{nat}}$ is its natural-spline subspace, with $f'' = 0$ at both boundary knots (dimension n). The existing implementation minimizes J over $\mathcal{S}_{\text{nat}}$; the new path minimizes J over $\mathcal{S}_{\text{spl}}$, a strictly *more general* problem, with no boundary conditions imposed. In general a larger search set could find a strictly smaller minimum. But a classical result (Wahba, 1990) states that the minimizer over all of $\mathcal{F}$ is a natural cubic spline with knots at the data:

$$\hat{f} := \arg \min_{f \in \mathcal{F}} J(f) \in \mathcal{S}_{\text{nat}}. \quad (19)$$

By (18), $\hat{f}$ lies in every one of the three sets, so minimizing over any of them returns the same function:

$$\arg \min_{\mathcal{S}_{\text{nat}}} J = \arg \min_{\mathcal{S}_{\text{spl}}} J = \hat{f}. \quad (20)$$

The natural boundary behaviour is not imposed on the new path; it *emerges* from the minimization, because end curvature costs penalty with no data beyond the boundary to justify it. Equation (20) is what the test verifies: the two implementations agree to machine precision at every tested λ (10^{-14} at $\lambda = 10^{-4}$, degrading only with conditioning to 10^{-9} at $\lambda = 100$), and any discrepancy beyond rounding would indicate a bug. This single test certifies the design matrix, the penalty (11), and the normal-equation solve simultaneously, with no external oracle.

The `fda` package was used strictly as a black-box oracle: only its numerical output was compared against; its GPL-licensed source was not consulted.

10 Conditioning at large λ

The question: the equivalence test of Section 9 agrees to 10^{-14} at small λ but only 10^{-9} at large λ . Why?

Symbols: κ the condition number (defined in Step 1), and $\varepsilon \approx 10^{-16}$ the machine precision.

Step 1: solving a linear system always loses digits. A computer carries about 16 digits ($\varepsilon \approx 10^{-16}$). When it solves $A\mathbf{c} = \mathbf{b}$, rounding errors get amplified by the condition number of A , and the answer is reliable only to

$$\text{error} \approx \varepsilon \cdot \kappa(A). \quad (21)$$

$\kappa(A)$ is the ratio of the largest to the smallest eigenvalue: how unevenly the matrix treats different directions. A matrix that is very stiff in some directions and very soft in others is the hard case.

Step 2: our matrix becomes exactly that as λ grows. The system we solve is

$$A_\lambda = X^\top W X + \lambda \Omega. \quad (22)$$

The second term grows with λ , but not in every direction, because Ω is blind to straight lines:

$$\Omega \mathbf{c}_{\text{line}} = \mathbf{0} \quad (\text{lines have no curvature}). \quad (23)$$

So in two directions of coefficient space (slope and intercept) the matrix keeps its fixed, λ -independent size, set by the data term alone, while in all $m - 2$ others it grows like λ . Largest over smallest:

$$\kappa(A_\lambda) \propto \lambda. \quad (24)$$

Step 3: combine (21) and (24) and test. The prediction is $\text{error} \propto \lambda$, lockstep with κ , decade for decade. Measured (against a `float80` iterative-refinement reference; same data and knots throughout):

λ	$\kappa(A_\lambda)$	this implementation	<code>fda::smooth.basis</code>
10^{-4}	$5 \cdot 10^3$	$5 \cdot 10^{-16}$	$2 \cdot 10^{-15}$
10^{-2}	$5 \cdot 10^4$	$2 \cdot 10^{-15}$	$1 \cdot 10^{-13}$
1	$4 \cdot 10^6$	$4 \cdot 10^{-13}$	$6 \cdot 10^{-12}$
10^2	$4 \cdot 10^8$	$8 \cdot 10^{-11}$	$1 \cdot 10^{-9}$
10^4	$4 \cdot 10^{10}$	$2 \cdot 10^{-8}$	$3 \cdot 10^{-8}$

The answer. κ climbs seven decades and the error climbs seven decades with it, eq. (21) in action. The degradation is not two methods disagreeing; it is every solver losing the same digits to the same κ . The reference implementation of Ramsay and Silverman (2005) follows the same line, slightly above ours at every λ : the difficulty is intrinsic to the problem. It is also harmless, λ large enough to cost 10^{-8} produces a fit already indistinguishable from its straight-line limit.

11 Related software

A survey of what neighbouring ecosystems provide, and where the proposed implementation sits. The three Julia packages were inspected at source level; `fda` was compared numerically only (Section 10), its GPL source unread.

package	knots	penalty & construction	λ selection	solve
<code>scipy</code> ($t=\text{None}$)	data only	$\int (f'')^2$, divided diff.	GCV	banded LU
<code>SmoothingSplines.jl</code>	data only	$\int (f'')^2$, Reinsch	manual	banded Cholesky
<code>BSplineKit.jl</code>	data only	$\int (f'')^2$ exact, value-space	manual	banded Cholesky
<code>GeneralizedSS.jl</code>	data only	$\int (f^{(p)})^2$, RKHS kernel	marginal lik.	semisep. Cholesky
<code>fda</code> (R)	free	$\int (f'')^2$ exact	GCV	normal eqs.
<code>mgcv</code> (R)	free	O'Sullivan / P-spline	GCV / REML	reparam.
this work	free	$\int (f'')^2$ exact, $\Omega = C^\top R C$	explicit λ	banded Cholesky

Three takeaways from the survey.

1. *Nobody outside R offers free knots.* The three Julia packages fix the knots at the data. `SmoothingSplines.jl` is the classical Reinsch algorithm (Reinsch, 1967). `GeneralizedSS.jl` uses a kernel formulation with no B-splines at all; its “generalized” refers to higher-order penalties and shape constraints (monotonicity, convexity), not to knots. So the combination this work adds, free knots with an exact integral penalty, outside `R`, exists nowhere in the surveyed ecosystems.

2. *Everybody solves the way we do.* Each package factors a symmetric positive-definite system: banded Cholesky in `SmoothingSplines.jl` (LAPACK `pbtrf`, the same routine under `solveh_banded`), semiseparable Cholesky in `GeneralizedSS.jl`; `fda`’s documentation formulates the solve as normal equations (Ramsay and Silverman, 2005), and its numerical behaviour is consistent with that (Section 10), though, as that section shows, conditioning alone cannot distinguish the formulation. None of the source-inspected packages uses QR.

3. *The same mass matrix everywhere.* The tridiagonal matrix of eqs. (8)–(9), diagonal $(h + h')/3$, off-diagonal $h/6$, appears verbatim as `ReinschR` in `SmoothingSplines.jl`, and (in its interior rows) as the grid-step matrix $\Delta/6$ in `BSplineKit.jl`’s smoothing fit, whose boundary rows are dropped because its natural boundary conditions zero the corresponding entries of γ . Both packages apply it to second-derivative *values* at the knots, where this document applies it to hat-basis *coefficients*, the same numbers, by the interpolatory property of Section 3, since their knots sit at the data. Reinsch’s 1967 algorithm and every implementation surveyed here share this single core object with eq. (11).

References

- C. de Boor. *A Practical Guide to Splines*, revised edition. Springer, New York, 2001.
- C. de Boor. B(asic)-spline basics. Technical report, 1986. <https://ftp.cs.wisc.edu/Approx/bsplbasic.pdf>
- J. O. Ramsay and B. W. Silverman. *Functional Data Analysis*, 2nd edition. Springer, New York, 2005. Ch. 5, “The roughness penalty approach.”
- M. P. Wand and J. T. Ormerod. On semiparametric regression with O’Sullivan penalized splines. *Australian & New Zealand Journal of Statistics*, 50(2):179–198, 2008.
- N. M. Patrikalakis, T. Maekawa, and W. Cho. *Shape Interrogation for Computer Aided Design and Manufacturing*. Hyperbook edition. <https://web.mit.edu/hyperbook/Patrikalakis-Maekawa-Cho/>
- G. Wahba. *Spline Models for Observational Data*. SIAM, Philadelphia, 1990.
- H. J. Woltring. A Fortran package for generalized, cross-validatory spline smoothing and differentiation. *Advances in Engineering Software*, 8(2):104–113, 1986.
- E. Zemlyanoy. Generalized cross-validation smoothing splines. BSc thesis, 2022. In Russian. <https://www.hse.ru/ba/am/students/diplomas/620910604>
- C. H. Reinsch. Smoothing by spline functions. *Numerische Mathematik*, 10(3):177–183, 1967.
- P. J. Green and B. W. Silverman. *Nonparametric Regression and Generalized Linear Models: A Roughness Penalty Approach*. Chapman & Hall/CRC, 1993.